Full Stack Assignment – 47

Assignment ID: FSD-47
Title: Conflict-Resolved Data Update System
Duration: 3 Hours

1. Objective

Design and develop a **Conflict-Resolved Data Update System** that handles concurrent updates to the same record by different users.
The system must detect update conflicts and enforce a consistent resolution strategy while ensuring **data persistence using a database**.

2. User Roles

USER

- Register and log in to the system
- View and update shared records

ADMIN

- Register and log in to the system
- View all records and conflict history

3. Functional Requirements

Authentication & Authorization

- Implement user registration and login
- Use JWT-based authentication
- Protect all backend APIs using JWT
- Enforce role-based access control on APIs and frontend views

Record Management

- Shared records must be stored in the database
- Users can update record fields
- Each update must include a version or timestamp

Conflict Detection

- System must detect conflicts when:
  - A record has been modified since it was last read by the user
- Conflict details must be stored in the database

Conflict Resolution

- System must apply a defined strategy:

    - Reject conflicting updates and require retry
- Users must be informed of conflicts clearly

4. Business Rule (Critical)

If a record has been updated by another user after it was last read, any subsequent update attempt must be rejected and marked as a conflict.
Silent overwrites are not allowed.

5. Technical Constraints (Mandatory)

- All record and conflict data must persist in a database
- In-memory or hardcoded conflict handling is not allowed
- Backend APIs must reflect actual database state
- JWT authentication and role validation are mandatory
- Use environment variables for sensitive configuration

6. Frontend Requirements

- Login and registration pages
- Record listing and edit view
- Clear conflict error messages on failed updates
- Conflict history view for ADMIN

UI design is not evaluated; correctness of conflict handling is the priority.

7. Submission Requirements (Strict)

GitHub Repository

- Backend source code
- Frontend source code
- Proper folder structure
- Clear and meaningful commit history

Live Deployment

- Live frontend deployment link
- Live backend deployment link
- Backend must be connected to the database

README.md (Mandatory)

The README file must include:

- Project overview
- Tech stack used
- User roles and permissions
- API endpoints

- Database schema or collections
- Steps to run the project locally
- Live frontend and backend links

8. Evaluation Note

Evaluation focuses on **conflict detection accuracy, consistency enforcement, database usage, and end-to-end execution**.